Name: Abderrahmane RHANDOURI
Course & Term: CS 428 – Cyber Capstone, Fall 2026
Team & Project: Group 4 – Alert Enrichment & Triage Automation Engine
Role: Project Manager / OSINT & Infrastructure Track Lead
Date: November 1, 2026
Reporting Period: Week 5 (Planning & Architecture Sprint)

What did you work on this week?

This week, I met with Jordan to discuss our project and determine how much access we could obtain to Kibana APIs and production data to feed into our engine. Jordan confirmed that we cannot use their Kibana APIs or real data; however, she suggested spinning up our own Kibana instance to showcase the project and build our minimum viable product (MVP).

I also built out our required infrastructure on Amazon Web Services (AWS). I provisioned a VPC with a dual-stack public subnet (IPv4/IPv6) hosting a jump server, which connects to a private, IPv6-only subnet where our application runs. The entire infrastructure was provisioned using Terraform, and all dependencies and tools were configured via Ansible—including a k3s Kubernetes cluster for container orchestration. The IPv6-only subnet connects to an egress-only internet gateway, making it reachable exclusively through the jump server.

What will you be working on for the next week?

Next week, I plan to research deployment options for hosting our Kibana instance in the cloud to accurately simulate a production environment.

I will also create architecture diagrams to clearly illustrate our infrastructure setup. Additionally, I will update our GitHub repository by configuring CI/CD deployment pipelines and GitHub Actions workflows, allowing us to deploy automatically via repository pushes rather than manual SSH sessions. These pipelines will incorporate automated unit testing and basic secret-scanning/DLP checks to prevent leaking sensitive API keys or credentials. Finally, I will establish branch protection rules requiring at least one peer review before any pull request is merged and deployed.

What are your roadblocks?

Additional Workload: Setting up and configuring a standalone Kibana instance within a tight timeframe adds another project to our current workload.

IPv6-to-IPv4 Compatibility: To keep cloud costs minimal or free, we are using an IPv6-only subnet on AWS. However, this instance cannot directly reach IPv4-only public endpoints (such as GitHub) to pull packages and dependencies. To resolve this, we are considering two approaches:

- Configuring the jump server as an outbound proxy (NAT64/forward proxy) to route and translate requests to the IPv4 internet.
- Pre-packaging and downloading the required dependencies on the jump server (or defining them strictly within Ansible playbooks) and transferring them directly to the application server.

How have you contributed to the team?

I provisioned and configured our entire core infrastructure, created individual user accounts on both the jump and application servers, and provided the team with documentation and commands to generate their SSH key pairs for secure cloud access. The deployment environment is now ready for the team's development work.